

Class 2

Thursday, September 10

Prediction, Generalization, and the Bias–Variance Trade-Off

Upcoming

Reading (complete by Tuesday, September 15): [ISL textbook](#) Chapter 3

Video (optional) (complete by Tuesday, September 15): [3.1 Simple Linear Regression](#); [3.3 Multiple Linear Regression](#)

Homework (due Thursday, September 17 at 12:59 p.m.): Homework 1

Key takeaways

- Expected test error is a measure of generalization for prediction problems. Test error measures expected performance on new, unseen observations that are ‘like’ (but not the same as!) the training data.
- We usually can’t compute expected test error directly, but we can estimate it with test data that was not used to train the prediction method.
- Under squared loss, $f(x) = \mathbb{E}[Y \mid X = x]$ has the lowest test error (and is therefore ‘best’ in a certain sense). We usually don’t know f , and so try to estimate it with some $\hat{f}$.
- More flexible methods typically have lower bias and higher variance, while less flexible methods typically have higher bias and lower variance. For KNN, smaller values of K are more flexible. Good test-set performance requires balancing bias and variance.

Summary notes

Recap: Supervised Regression Data-Generating Process from Last Lecture. In statistics, we usually start with the data. In supervised learning problems, the training data are

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}.$$

We then tell a story about how the data came to be. In the first class, I referred to this as a model. But because the term model is over-loaded, moving forward I’ll try to use the phrase ‘data-generating process’, since the story tells us something about how the data could be generated. In lecture 1, we introduced the supervised regression data-generating process:

In the first class and in the textbook, we saw the following data-generating process, that we will continue to use.

$$Y = f(X) + \epsilon,$$

where f is fixed but unknown, X and ϵ are independent, and

$$\mathbb{E}[\epsilon] = 0.$$

This model says that the output can be obtained by taking the input, applying a function to it, then adding some noise.

Independence of X and ϵ implies $\mathbb{E}[\epsilon \mid X = x] = \mathbb{E}[\epsilon] = 0$, so

$$f(x) = \mathbb{E}[Y \mid X = x].$$

We will also often assume that all n observations are independent and identically distributed copies of (X, Y) .

Prediction Goal. We want to use the training data, $\mathcal{D}$, to learn a function $\hat{f}_{\mathcal{D}}$ such that

$$Y \approx \hat{f}_{\mathcal{D}}(X)$$

on *new* observations. We use the subscript to indicate our predictor depends on the training data we observed.

Measuring prediction quality with a loss function. A *loss function* or simply a *loss* $L(y, a)$ assigns a cost to predicting a when the observed response is y . Usually, a loss is large if a and y are very different, and close to 0 if they are similar. The choice of a loss function is in some sense a value judgement about what matters in a prediction problem. In practice (for better or worse), people often use squared-error loss as a ‘default’ in regression problems:

$$L(y, a) = (y - a)^2.$$

For regression problems in this course, we will mostly use squared loss, which is also the focus of the book. Squared loss is mathematically convenient, and at least has the property that it is 0 if and only if our predictions agree with the observed data, and larger otherwise. But other losses might be more appropriate for a particular application, and ideally a loss would reflect how predictions will be used in decision-making or some other downstream application.

What properties might you want from a loss function for regression?

Can you suggest a loss function for a classification problem where there are two possible categories? E.g. if the task is to classify whether an image is a 0 or a 1.

Training error. The *training error* of a prediction method is its average loss on the training data. For squared loss, this is also referred to as the *training mean squared error (training MSE)*, and is equal to:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{\mathcal{D}}(x_i))^2.$$

At first, this is an appealing measure of prediction quality: it is easy to compute, and it is small if the predictions are close to the observed responses. But training error is not a good measure of generalization, which is what we really care about. In fact, later today we will see a prediction method that has training MSE equal to 0 whenever the training inputs are distinct, but often generalizes poorly. This is an example of *overfitting*: fitting the training observations very closely without achieving comparably good performance on new observations. The reason training error is not a good measure of generalization is that it measures how well a prediction method fits data that it has already seen, but we care about how well it will perform on new data that it has not seen.

Test error and generalization. Let (X_0, Y_0) be a new observation drawn from the same population as the training data but independent of $\mathcal{D}$. Once a particular prediction method $\hat{f}_{\mathcal{D}}$ has been learned, its expected loss on this new observation is:

$$R(\hat{f}_{\mathcal{D}} \mid \mathcal{D}) = \mathbb{E}_{X_0, Y_0} [L(Y_0, \hat{f}_{\mathcal{D}}(X_0)) \mid \mathcal{D}].$$

This expectation describes average performance of a fitted prediction method on *unseen data*. Remember that the training data are i.i.d. copies of (X, Y) . Therefore, $R(\hat{f}_{\mathcal{D}} \mid \mathcal{D})$ is *random* because it depends on the random training data. We can also average over the training data to get the *expected test error*:

$$\mathbb{E}_{\mathcal{D}} [R(\hat{f}_{\mathcal{D}} \mid \mathcal{D})] = \mathbb{E}_{\mathcal{D}, X_0, Y_0} [L(Y_0, \hat{f}_{\mathcal{D}}(X_0))].$$

Expected test error is a measure of *generalization*: how well a prediction method learned from the training data performs on new observations that look like the training data, in the sense that they are drawn from

the same population. A method generalizes well when patterns learned from the training sample continue to give low loss on these new observations.

With squared loss, expected test error is referred to as *test mean squared error*.

The textbook uses *test MSE* for the empirical mean squared error computed on a *particular* test set, which we discuss next.

Estimating expected test error with a test set. We usually can't compute the expected test error directly, because we need to average over all possible new observations. But if we have an independent *test set* that was not used to train the model, we can estimate test error by computing the average loss on this test set. In particular, if we have m test observations $(x_1^{\text{test}}, y_1^{\text{test}}), \dots, (x_m^{\text{test}}, y_m^{\text{test}})$, that are all independent of the training data and each other, and have the same distribution as the training data, then under squared loss the test MSE is

$$\frac{1}{m} \sum_{i=1}^m (y_i^{\text{test}} - \hat{f}_{\mathcal{D}}(x_i^{\text{test}}))^2.$$

If the test set is used to fit the model (e.g., to choose hyperparameters), then the test MSE is no longer an unbiased estimate of expected test error.

Optimal prediction under squared loss. What is the best possible prediction we can make, according to square loss? To answer this question, let's condition on a particular (new) input X_0 , and imagine that we predict some number, $c = \hat{f}_{\mathcal{D}}(X_0)$. The optimal value of c is

$$\arg \min_{c \in \mathbb{R}} \mathbb{E}[(Y_0 - c)^2 | X_0] \quad (1)$$

Recall from our data generating process that $Y_0 = f(X_0) + \epsilon$. And we might think that a reasonable prediction for $Y_0 | X_0$ is $f(X_0)$, since we think of the other term as an error term. This suggests plugging in the above formula, and grouping together $f(X_0)$ and c .

$$\arg \min_{c \in \mathbb{R}} \mathbb{E}[(Y_0 - c)^2 | X_0] = \arg \min_{c \in \mathbb{R}} \mathbb{E}[(\epsilon + (f(X_0) - c))^2 | X_0]. \quad (2)$$

Following the somewhat lengthy algebra we saw in class,

$$(\epsilon + (f(X_0) - c))^2 = \epsilon^2 + (f(X_0) - c)^2 + 2\epsilon(f(X_0) - c). \quad (3)$$

Taking a conditional expectation on both sides and using linearity of expectation,

$$\mathbb{E}[(\epsilon + (f(X_0) - c))^2 | X_0] = \mathbb{E}[\epsilon^2 + (f(X_0) - c)^2 + 2\epsilon(f(X_0) - c) | X_0] \quad (4)$$

$$= \mathbb{E}[\epsilon^2 | X_0] + \mathbb{E}[(f(X_0) - c)^2] + \mathbb{E}[2\epsilon(f(X_0) - c) | X_0]. \quad (5)$$

We now consider the first term. Because $\mathbb{E}[\epsilon] = 0$ by our data generating process assumption, and because ϵ is independent of X_0 ,

$$\mathbb{E}[\epsilon^2 | X_0] = \mathbb{E}[\epsilon^2] = \text{Var}(\epsilon). \quad (6)$$

We next consider the last (cross-term). Because c can only depend on X_0 (and not on ϵ), $2(f(X_0) - c)^2 | X_0$ can be pulled out of the expectation

$$\mathbb{E}[2\epsilon(f(X_0) - c) | X_0] = 2(f(X_0) - c)\mathbb{E}[\epsilon | X_0] = 2(f(X_0) - c)\mathbb{E}[\epsilon] = 0. \quad (7)$$

Combining the various pieces above,

$$\mathbb{E}[(Y_0 - c)^2 | X_0] = \text{Var}(\epsilon) + \mathbb{E}[(f(X_0) - c)^2 | X_0]. \quad (8)$$

Now we return to the question of what c minimizes this squared loss (conditional on X_0). We see that the first term doesn't depend at all on c , and so any predictor has expected test loss at least equal to $\text{Var}(\epsilon)$.

On the other hand, the second term is non-negative, and is equal to 0 if and only if $c = f(X_0) = \mathbb{E}[Y_0|X_0]$. Therefore, the *optimal* prediction rule for squared loss is to, given a new X_0 , predict $\mathbb{E}[Y_0|X_0]$. The issue is, we usually don't know this conditional expectation! And so the goal of regression (at least for squared loss), is to estimate the conditional expectation of a new output, given a corresponding new input based on the training data available to us.

Optimal Prediction For 0 – 1 Loss Classification. During lecture, we saw the *classification* loss

$$L(y, a) = \begin{cases} 1 & y \neq a, \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

The optimal prediction rule for this loss and in the classification problem is no longer the conditional expectation. Instead, it is the predictor that predicts the most probably class of Y given the observed value of X . You will need this fact on the homework. This predictor is referred to as the Bayes Classifier, and (in equation form) predicts:

$$\arg \max_{j \in \{1, \dots, K\}} \Pr(Y_0 = j | X_0 = x), \quad (10)$$

where Y takes values among K classes, and x is the new value at which we want to make predictions. The *Bayes decision boundary* is the part of input space over which there are multiple classes that are the most likely class — for example when only two classes are possible, the Bayes decision boundary corresponds to inputs for which both class labels are equally likely. See section 2.2.3 in ISLR for more details.